Niklas Glaser

Heinrich-Heine-Universität Düsseldorf

April 2026

Overview

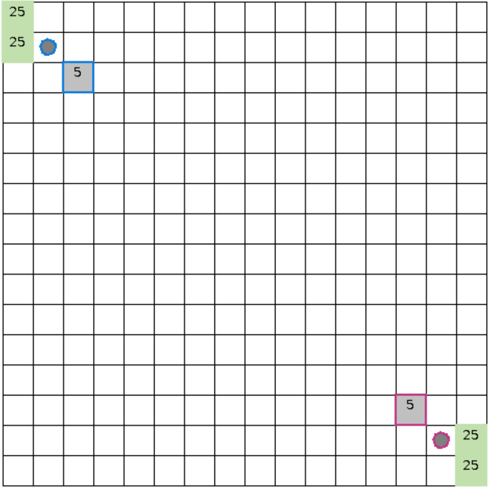
Problem Setting

Why MicroRTS?

- simplified RTS environment for RL research
- still contains resource management, unit production and combat
- supports controlled and repeatable experiments

Main challenge

- large combinatorial action space
- simultaneous decisions
- delayed and sparse rewards
- robustness against diverse opponents



Initial state of the `basesWorkers16x16A` map.

Motivation and Research Question

Starting point

The existing Base Agent trained with Behavioral Cloning (BC) and Proximal Policy Optimization (PPO) is already strong, but it still has weaknesses against several opponents and a limited strategic repertoire.

Goal

Improve robustness and generalization beyond a fixed set of training bots by adding Self-Play and League Training (LT).

Research question

How effective is League Training with PPO in improving the robustness and generalization of a BC/PPO-pretrained MicroRTS agent, and how strongly does this depend on the diversity of the initial agents and BC experts?

Training Pipeline

$$\text{BC} \rightarrow \text{BC-FT} \rightarrow \text{PPO} \rightarrow \text{LT} \rightarrow \text{PPO fine-tuning}$$

- **BC / BC-FT:** imitate strong scripted opponents to obtain a competent initial policy
- **PPO:** refine the policy through exploration
- **LT:** train against a population of evolving opponents instead of only bots or pure self-play
- **Fine-tuning:** fix remaining weaknesses after LT, especially against WorkerRushAI

From Self-Play to League Training

SP	train against the current version of the agent; simple, but prone to overfitting or cyclical strategies
FSP	train against a set of past policies; reduces forgetting, but opponent choice is still crude
PFSP	sample opponents based on current win rates; focuses on informative and non-trivial matchups
LT	add role-based agents: Main Agent, Historical Agents, Main Exploiters and League Exploiters

Why LT?

LT does not only preserve diversity through historical snapshots, it also creates diversity through exploiters that actively search for weaknesses of the current policy.

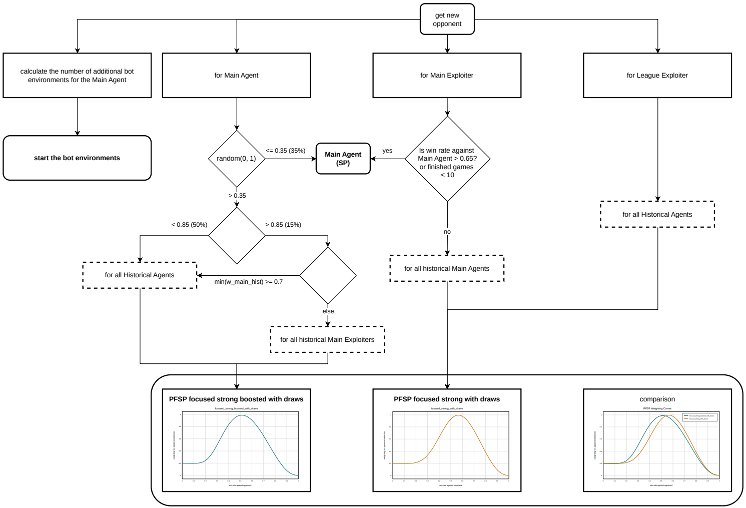
League Training Design

- **Main Agent:** primary policy
- **Historical Agents:** frozen snapshots for stability
- **Main Exploiters:** exploit weaknesses of historical Main Agents
- **League Exploiters:** search for broader league weaknesses
- **PFSP:** opponent sampling based on current win rates

Key idea

LT turns robustness against diverse opponents into part of the optimization target.

League Training Matchmaking

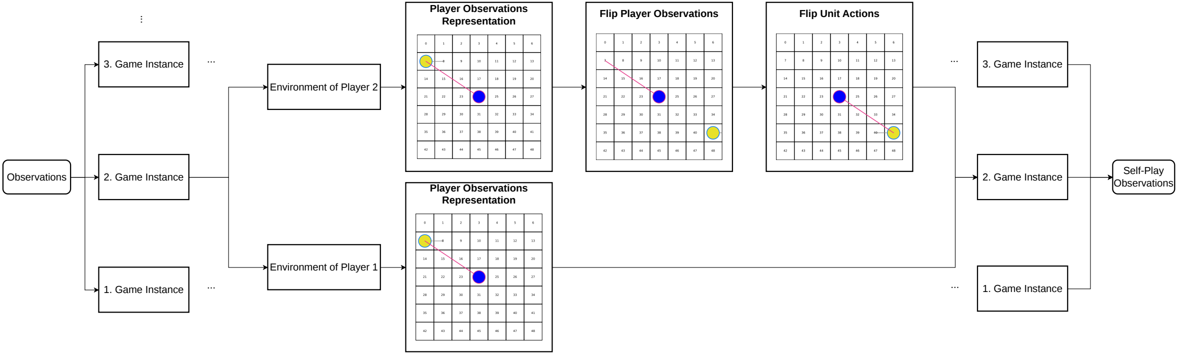


Matchmaking and opponent sampling in the League Training setup.

Important Implementation Changes

- dynamic bot environments for CoacAI and Mayari to stabilize the early LT phase
- observation adjustment for player 2 in self-play by flipping GridNet observations
- delta score reward to provide denser learning signals in MicroRTS
- annealed entropy coefficient to increase early exploration, especially for exploiters
- new initial exploiter agents with different unit-type focuses to improve diversity
- corrected exploiter reset and checkpoint logic compared to the AlphaStar pseudocode

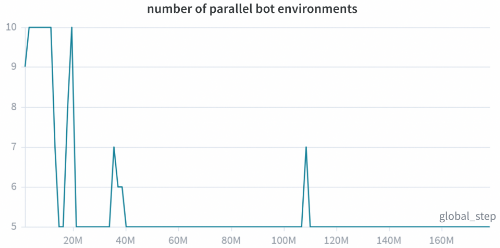
Observation Adjustment for Self-Play

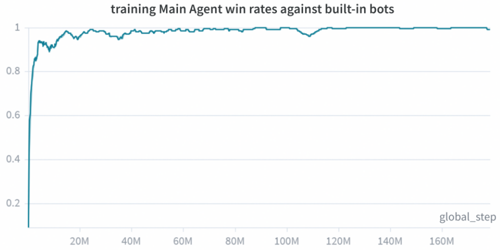


Player-2 observations are transformed so that self-play remains compatible with a Base Agent trained only as player 1.

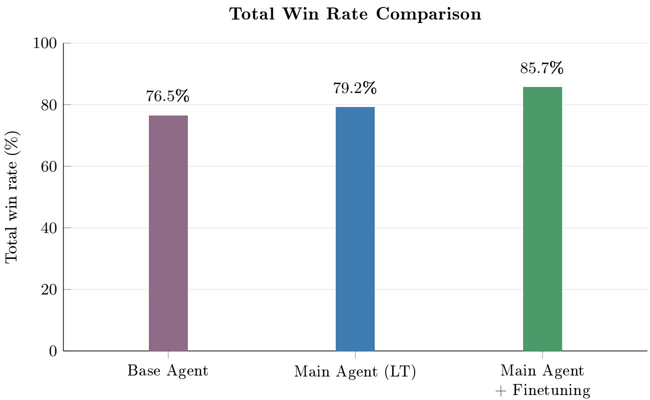
Experiment Setup

- map: `basesWorkers16x16A`
- LT setup: 1 Main Agent, 4 Main Exploiters, 1 League Exploiter
- 25 environments per learning agent
- 5 to 10 additional dynamic bot environments
- approximately 130 million LT steps
- training time: about 100 hours
- maximum setup size: 160 parallel environments
- high memory cost, CPU bottleneck





Main Quantitative Result



- overall win rate improves from **76.5%** to **79.2%** after LT
- after PPO fine-tuning it reaches **85.7%**
- overall loss rate drops from **7.7%** to **2.2%**

Evaluation Results

Opponent	Main Agent without unit focus	Base Agent	Main Agent (LT)	Main Agent + Fine-tuning
bots used in training				
CoacAI	**100.0**	96.0	**100.0**	**100.0**
Mayari	**100.0**	67.3	**100.0**	**100.0**
Built-in bots				
WorkerRushAI	31.0	**100.0**	35.0	99.5
PassiveAI	**100.0**	99.7	**100.0**	**100.0**
LightRushAI	**100.0**	98.0	**100.0**	**100.0**
RandomAI	**100.0**	99.0	**100.0**	**100.0**
RandomBiasedAI	91.0	79.0	97.0	**97.5**
Other bots				
Rojo	**100.0**	81.0	**100.0**	**100.0**
MixedBot	45.0	43.3	57.0	**60.5**
Izanagi	68.0	71.7	70.0	**72.5**
Tiamat	94.0	93.0	**99.0**	**99.0**
Droplet	45.0	67.7	62.0	**75.5**
GuidedRojoA3N	64.0	74.7	87.0	**88.5**
NaiveMCTS	0.0	0.7	2.0	**6.5**
overall	74.1	76.5	79.2	**85.7**

Evaluation Results

Opponent	Main Agent without unit focus	Base Agent	Main Agent (LT)	Main Agent + Fine-tuning
bots used in training				
CoacAI	**0.0**	2.0	**0.0**	**0.0**
Mayari	**0.0**	27.3	**0.0**	**0.0**
Built-in bots				
WorkerRushAI	63.0	**0.0**	55.0	0.5
PassiveAI	**0.0**	**0.0**	**0.0**	**0.0**
LightRushAI	**0.0**	2.0	**0.0**	**0.0**
RandomAI	**0.0**	**0.0**	**0.0**	**0.0**
RandomBiasedAI	**0.0**	**0.0**	**0.0**	**0.0**
Other bots				
Rojo	**0.0**	5.3	**0.0**	**0.0**
MixedBot	33.0	34.3	20.0	**13.5**
Izanagi	15.0	16.3	7.0	**6.0**
Tiamat	6.0	7.0	1.0	**0.5**
Droplet	22.0	**9.0**	22.0	10.5
GuidedRojoA3N	**0.0**	1.7	**0.0**	**0.0**
NaiveMCTS	**0.0**	2.7	**0.0**	**0.0**
overall	9.9	7.7	7.5	**2.2**

Interpretation of the Results

hypothesis:

- LT turns robustness against diverse opponents into part of the optimization target
- LT helps to find a better local optimum that is then further improved by fine-tuning, while pure PPO often gets stuck in not as generalized local optima.

Player 1 Priority

Observation

MicroRTS gives player 1 action priority, so a potential concern is that self-play style training could accidentally favor player-1-specific behavior.

Result from the thesis

In one self-match evaluation of the same agent over 200 games, player 1 achieved 38% wins, 28% draws and player 2 achieved 34% wins.

- minimal effect onevaluations
- alternating the learning side to prevent the agent from exploiting player-1-specific priority

▸ Open Player 1 Priority Example Video

Main Limitation: Diversity

Base Agent vs Base Agent

Video example for the Base Agent's preferred strategy

Open Video

Main Limitation: Diversity

- Exploiters initialized from similar policies tend to discover similar weaknesses.
- Some strategically different exploits are hard to reach because they require first unlearning the Base Agent's preferred behavior.
- New initial agents with focuses on different unit types to improve exploiter diversity and Main Agent generalization.
- Hypothesis: LT therefore depends strongly on the diversity already present at initialization

Scope and Remaining Limitations

- evaluation is limited to one map: `basesWorkers16x16A`
- the setting uses full observability and no fog of war
- LT requires substantially more computation than pure PPO
- computation
- exploiter diversity is still the main bottleneck

Takeaway

The method works, but the strength of the final result depends strongly on the diversity and cost of the training setup.

Conclusion

Conclusion

League Training with PPO improves the robustness and generalization of the MicroRTS Base Agent. Additional PPO fine-tuning improves the final agent even further.

Key takeaway

The effectiveness of LT depends strongly on exploiter diversity, which in turn depends on the diversity of the initial agents and BC experts.

- LT improves most matchups and the overall performance
- fine-tuning further improves the final agent
- diversity of initial agents is the critical practical factor

Future Work

- train on multiple maps and with fog of war
- improve the diversity and strength of initial agents
- reduce computational cost of MicroRTS training
- investigate stronger diversity mechanisms for exploiters

Thank you

Questions?